Expt 7: Implementation of Travelling Salesperson Problem using heuristic approach

7.1. PROBLEM SPECIFICATION

To implement and demonstrate a heuristic approach for solving the Travelling Salesperson Problem (TSP) — the classical NP-hard combinatorial optimization problem in which a salesperson, given a set of n cities and the pairwise distances (or costs) between them, must determine the shortest possible route that:

(i) starts from a designated source city,

(ii) visits every other city exactly once, and

(iii) returns to the source city,

such that the total distance (tour cost) travelled is minimized.

Specific objectives of the experiment:

1. To formulate the TSP as a search/optimization problem by representing the cities and inter-city distances as a weighted, fully connected graph (cost matrix).

2. To understand why an exhaustive (brute-force) search over all (n − 1)!/2 possible tours becomes computationally infeasible as n grows, motivating the need for heuristic methods.

3. To design and implement a heuristic algorithm (e.g., Nearest Neighbour, Greedy edge selection that produces a near-optimal tour in reasonable time.

4. To trace the construction of the tour step by step, recording the sequence of cities visited and the cumulative cost at each step.

5. To compute and report the final tour and its total cost, and to compare the heuristic solution against the optimal (or known best) solution where feasible, thereby analysing the quality–efficiency trade-off of heuristic search.

7.2. Heuristic Functions for TSP

Since the Travelling Salesperson Problem is NP-hard, exact algorithms become impractical for large n. Heuristic functions are therefore used to guide the search toward a good (near-optimal) tour in reasonable time. A heuristic function h(state) estimates the desirability — typically the remaining tour cost — of a partially constructed tour, and is used to decide which city to visit next or which candidate tour to retain.

Commonly used heuristics for TSP:

1. **Nearest Neighbour Heuristic (NNH)**: From the current city, always move to the nearest unvisited city.

$$h(c) = min\{ d(current, c) : c \notin visited \}.$$

Simple and fast ($O(n^2)$), but can be misled by greedy choices and typically yields tours within 25% of optimal.

2. **Greedy Edge Heuristic**: Sort all edges in non-decreasing order of weight and repeatedly add the shortest edge that neither creates a cycle of length < n nor causes any vertex to have degree > 2. Produces a single Hamiltonian cycle made of globally short edges.

3. **Cheapest Insertion Heuristic**: Start with a small sub-tour (e.g., a triangle of three cities). At each step, insert the unvisited city c at the position (i, j) in the current tour that minimises the insertion cost

$$d(i,c) + d(c,j) - d(i,j).$$

4. **Minimum Spanning Tree (MST) Heuristic**: Build an MST of the city graph, perform a pre-order traversal of the MST, and shortcut repeated vertices to obtain a Hamiltonian tour. Guarantees a tour of cost ≤ 2 × optimal for metric TSP.

5. **Christofides' Heuristic**: An MST-based construction combined with a minimum-weight perfect matching on odd-degree vertices. Guarantees a tour ≤ 1.5 × optimal for metric TSP.

6. **Metaheuristic Improvement (2-opt / 3-opt, Hill Climbing, Simulated Annealing, Genetic Algorithm)**: Start from any initial tour (often produced by NNH) and iteratively improve it by swapping edges or recombining tours. The heuristic h here is the total tour cost, which the search tries to minimise.

In this experiment, the Nearest Neighbour heuristic is used as the primary construction heuristic, optionally followed by a 2-opt improvement step to refine the tour.

7.3. Implementing TSP using simulated annealing

Simulated Annealing (SA) is a probabilistic local-search metaheuristic inspired by the physical annealing of metals: a material is heated and then cooled slowly so its atoms settle into a low-energy crystalline state. By analogy, SA explores the space of TSP tours by occasionally accepting worse tours — with a probability that decreases as a control parameter T ("temperature") is lowered — thereby escaping local minima that pure hill-climbing or 2-opt would get trapped in.

Problem mapping (TSP → SA):

• State: a candidate Hamiltonian tour, represented as a permutation of the n cities (with the start city fixed).

• Energy / cost E(s): total length of the tour s, i.e. the sum of inter-city distances including the return edge to the start.

• Neighbour s′: a tour obtained from s by a small random perturbation — typically a 2-opt swap (reverse a randomly chosen sub-segment of the tour) or a random pair-swap of two cities.

• Acceptance probability: if $\Delta E = E(s') - E(s)$,

– if $\Delta E < 0$, accept s′ (it is shorter);

– else accept s′ with probability $P = \exp(-\Delta E / T)$.

· Cooling schedule: T is reduced over iterations, e.g. geometric cooling T ← α · T with 0.8 ≤ α < 1, until T reaches a small final temperature T_min or no improvement is seen for a number of iterations.

7.4. Algorithm: TSP_Simulated_Annealing

Input:

- distance matrix $d[1..n][1..n]$,
- initial temperature T_0,
- final temperature T_{min},
- cooling rate α,
- iterations per temperature L

Output

- best tour found and its cost

1. Construct an initial tour s (random permutation, or Nearest Neighbour tour).

2. best ← s;

   T ← T_0.

3. while T > T_{min}:

   for k = 1 to L:

   a. Generate neighbour s′ by applying a 2-opt move (pick i < j, reverse s[i..j]).

   b. Compute $\Delta E = cost(s') - cost(s)$.

   c. if $\Delta E < 0$ then s ← s′

   else generate r ∈ [0,1) uniformly;

   if $r < e^{(-\Delta E / T)}$ then s ← s′.

   d. if cost(s) < cost(best) then best ← s.

   T ← α · T.

4. return best, cost(best).

Why it works for TSP:

- At high T, $e^{(-\Delta E / T)} \approx 1$, so almost every neighbour is accepted — the search behaves like a random walk and explores the tour space broadly.
- As T decreases, the probability of accepting a worsening move shrinks, so the search increasingly behaves like hill-climbing and converges to a low-cost tour.
- The occasional uphill moves let the search escape local minima where 2-opt or Nearest Neighbour would otherwise stall.

Typical parameter choices:

· T_0: set so that the initial acceptance ratio of worsening moves is around 0.8 (e.g., T0 = average $|\Delta E|$ / $\ln(0.8^{-1})$).

• α: 0.90–0.99 (slower cooling → better tours, more time).

· L (Markov-chain length per temperature): O(n) or $O(n^2)$.

· T_{min}: 10^{-3} to 10^{-6}, or stop when no improvement for K consecutive temperatures.

Complexity and quality:

- each iteration costs O(1) to O(n) depending on whether the tour cost is recomputed incrementally;
- total cost is O(L · number_of_temperatures).
- For metric TSP, a well-tuned SA typically returns tours within 1–3% of optimal for n up to a few hundred cities, and combines well with a final 2-opt or 3-opt polish.

7.5. PROGRAM

https://github.com/mayurjambit/BAD402

7.6. OUTPUT